

```
import io
import json
import os
import time
import streamlit as st
from google import genai
from google.genai import types
from pydantic import BaseModel, Field
from pypdf import PdfReader
from reportlab.lib import colors
from reportlab.lib.pagesizes import letter
from reportlab.lib.styles import ParagraphStyle, getSampleStyleSheet
from reportlab.platypus import Paragraph, SimpleDocTemplate, Spacer, Table, TableStyle
```

```
# Page configuration
```

```
st.set_page_config(page_title="Civil Services Mock Portal", page_icon="📝",
layout="centered")
```

```
STORAGE_FILE = "active_test.json"
```

```
ADMIN_PASSWORD = os.getenv("ADMIN_PASSWORD", "admin123")
```

```
# --- 1. Data Schema ---
```

```
class QuizQuestion(BaseModel):
```

```
    question: str = Field(description="The question prompt")
```

```
    options: list[str] = Field(description="Exactly 4 choices")
```

```
    correct_index: int = Field(description="0-based index of correct option (0, 1, 2, or 3)")
```

```
    explanation: str = Field(description="Analytical explanation of the answer")
```

```
class ExamPaper(BaseModel):
```

```
    title: str = Field(description="Title of the mock test")
```

```
    questions: list[QuizQuestion]
```

```
# --- 2. Helper Functions ---
```

```
def load_published_test():
```

```
    if os.path.exists(STORAGE_FILE):
```

```
        try:
```

```
            with open(STORAGE_FILE, "r", encoding="utf-8") as f:
```

```
                return json.load(f)
```

```
        except Exception:
```

```
            return None
```

```
    return None
```

```
def save_published_test(data: dict):
```

```
    with open(STORAGE_FILE, "w", encoding="utf-8") as f:
```

```
        json.dump(data, f, indent=2, ensure_ascii=False)
```

```
def extract_text_from_pdf(pdf_file) -> str:
```

```
    reader = PdfReader(pdf_file)
```

```

text = ""
for page in reader.pages:
    extracted = page.extract_text()
    if extracted:
        text += extracted + "\n"
return text

def parse_pdf_to_exam(api_key: str, text: str, count: int, title: str) -> dict:
    client = genai.Client(api_key=api_key)
    prompt = f"""
    You are an expert civil services exam paper creator.
    Frame {count} high-yield multiple-choice questions from the provided text.
    Create authentic statement-based options, ensure plausible distractors, and verify the
    correct choice.
    Set the title as: '{title}'

    Study Material:
    {text[:30000]}
    """
    response = client.models.generate_content(
        model="gemini-2.5-flash",
        contents=prompt,
        config=types.GenerateContentConfig(
            response_mime_type="application/json",
            response_schema=ExamPaper,
            temperature=0.2,
        ),
    )
    return json.loads(response.text)

def generate_pdf_report(title: str, total_q: int, correct: int, wrong: int, unattempted: int,
net_score: float, max_marks: float, questions: list, user_choices: dict) -> bytes:
    """Generates an exam scorecard PDF in memory."""
    buffer = io.BytesIO()
    doc = SimpleDocTemplate(buffer, pagesize=letter, rightMargin=36, leftMargin=36,
topMargin=36, bottomMargin=36)
    styles = getSampleStyleSheet()
    story = []

    # Title & Header
    title_style = ParagraphStyle('DocTitle', parent=styles['Heading1'], fontSize=18, leading=22,
textColor=colors.HexColor('#1E293B'))
    story.append(Paragraph(f"Scorecard: {title}", title_style))
    story.append(Spacer(1, 10))

    # Summary Table
    summary_data = [
        ["Total Questions", str(total_q), "Marks per Correct", "+2.00"],

```

```

["Attempted", str(correct + wrong), "Negative Marking", "-0.66 (1/3rd)",
["Correct Answers", str(correct), "Incorrect Answers", str(wrong)],
["Unattempted", str(unattempted), "Net Score", f"{net_score:.2f} / {max_marks:.2f}"]
]
summary_table = Table(summary_data, colWidths=[130, 130, 130, 130])
summary_table.setStyle(TableStyle([
    ('BACKGROUND', (0, 0), (-1, -1), colors.HexColor('#F8FAFC')),
    ('TEXTCOLOR', (0, 0), (-1, -1), colors.HexColor('#0F172A')),
    ('FONTNAME', (0, 0), (-1, -1), 'Helvetica-Bold'),
    ('FONTSIZE', (0, 0), (-1, -1), 9),
    ('GRID', (0, 0), (-1, -1), 0.5, colors.HexColor('#CBD5E1')),
    ('ALIGN', (1, 0), (1, -1), 'CENTER'),
    ('ALIGN', (3, 0), (3, -1), 'CENTER'),
]))
story.append(summary_table)
story.append(Spacer(1, 18))

# Detailed Q&A Section
h2_style = ParagraphStyle('H2', parent=styles['Heading2'], fontSize=13,
textColor=colors.HexColor('#0F172A'))
story.append(Paragraph("Detailed Question Analysis", h2_style))
story.append(Spacer(1, 8))

body_style = ParagraphStyle('Body', parent=styles['Normal'], fontSize=8.5, leading=12)
correct_style = ParagraphStyle('CorrectOpt', parent=styles['Normal'], fontSize=8,
leading=11, textColor=colors.HexColor('#166534'))
wrong_style = ParagraphStyle('WrongOpt', parent=styles['Normal'], fontSize=8,
leading=11, textColor=colors.HexColor('#991B1B'))
exp_style = ParagraphStyle('Exp', parent=styles['Italic'], fontSize=8, leading=11,
textColor=colors.HexColor('#334155'))

for i, q in enumerate(questions):
    user_ans = user_choices.get(i)
    is_correct = user_ans == q["correct_index"]
    status = "CORRECT (+2.0)" if is_correct else ("UNATTEMPTED (0.0)" if user_ans is
None else "WRONG (-0.66)")

    status_color = "#166534" if is_correct else ("#64748B" if user_ans is None else
"#991B1B")
    q_header = f"<b>Q{i+1}.</b> {q['question']} <font
color='{status_color}'><b>{status}</b></font>"
    story.append(Paragraph(q_header, body_style))
    story.append(Spacer(1, 3))

    for o_idx, opt in enumerate(q["options"]):
        opt_letter = chr(65 + o_idx)
        if o_idx == q["correct_index"]:

```

```

        story.append(Paragraph(f"&nbsp;&nbsp;<b>{opt_letter}</b> {opt} &#10004;
(Correct)", correct_style))
        elif o_idx == user_ans:
            story.append(Paragraph(f"&nbsp;&nbsp;<b>{opt_letter}</b> {opt} &#10008; (Your
Choice)", wrong_style))
        else:
            story.append(Paragraph(f"&nbsp;&nbsp;<b>{opt_letter}</b> {opt}", body_style))

    story.append(Spacer(1, 2))
    story.append(Paragraph(f"<b>Explanation:</b> {q['explanation']}", exp_style))
    story.append(Spacer(1, 8))

doc.build(story)
buffer.seek(0)
return buffer.getvalue()

```

--- 3. Sidebar: Admin Controls ---

with st.sidebar:

```

    st.header("Portal Navigation")
    mode = st.radio("Mode:", ["Student Practice", "Admin (Upload Test)"])

```

```

admin_authenticated = False

```

```

if mode == "Admin (Upload Test)":

```

```

    pwd = st.text_input("Admin Passcode", type="password")

```

```

    if pwd == ADMIN_PASSWORD:

```

```

        admin_authenticated = True

```

```

        st.success("Admin authorized.")

```

```

    elif pwd:

```

```

        st.error("Invalid passcode.")

```

--- 4. Admin View ---

```

if mode == "Admin (Upload Test)":

```

```

    if not admin_authenticated:

```

```

        st.info("Enter admin passcode to configure and publish new question papers.")

```

```

    else:

```

```

        st.title("📄 Publish Mock Paper")

```

```

        admin_key = st.text_input("Gemini API Key", type="password")

```

```

        exam_title = st.text_input("Exam Name", value="UPPSC / UPSC Current Affairs Mock")

```

```

        q_count = st.slider("Number of Questions", min_value=5, max_value=30, value=10)

```

```

        exam_duration = st.number_input("Time Limit (Minutes)", min_value=1,

```

```

max_value=180, value=15)

```

```

    pdf = st.file_uploader("Upload Notes/Paper (PDF)", type=["pdf"])

```

```

    if st.button("Generate & Publish Live", type="primary"):

```

```

        if not admin_key or not pdf:

```

```

            st.error("API KEY and PDF file are required.")

```

```

        else:

```

```

        with st.spinner("Extracting content and formatting questions..."):
            raw_text = extract_text_from_pdf(pdf)
            paper = parse_pdf_to_exam(admin_key, raw_text, q_count, exam_title)
            paper["time_limit_seconds"] = int(exam_duration * 60)
            save_published_test(paper)
            st.success(f"Exam '{exam_title}' is now live for all visitors!")

# --- 5. Student View ---
else:
    active_test = load_published_test()

    if not active_test or not active_test.get("questions"):
        st.title("📝 Civil Services Examination Portal")
        st.info("No mock test is currently active. Please await the administrator's upload.")
    else:
        questions = active_test["questions"]
        time_limit = active_test.get("time_limit_seconds", 900)
        exam_title = active_test.get("title", "Mock Examination")

# Session state setup
if "exam_active" not in st.session_state:
    st.session_state.exam_active = False
if "user_choices" not in st.session_state:
    st.session_state.user_choices = {}
if "start_time" not in st.session_state:
    st.session_state.start_time = None
if "exam_submitted" not in st.session_state:
    st.session_state.exam_submitted = False
if "q_idx" not in st.session_state:
    st.session_state.q_idx = 0

# Start Screen
if not st.session_state.exam_active and not st.session_state.exam_submitted:
    st.title(exam_title)

    c1, c2, c3 = st.columns(3)
    c1.metric("Questions", len(questions))
    c2.metric("Duration", f"{time_limit // 60} Mins")
    c3.metric("Marking Scheme", "+2.0 / -0.66")

    st.markdown("""
    **Exam Instructions:**
    * Each correct answer awards **+2.0 marks**.
    * Each incorrect answer incurs a penalty of **-0.66 marks** (1/3rd negative marking).
    * Unattempted questions carry **0 marks**.
    * The countdown timer starts immediately once you click **Begin Test**.
    """)

```

```

if st.button("Begin Test 🚀", type="primary"):
    st.session_state.exam_active = True
    st.session_state.start_time = time.time()
    st.session_state.user_choices = {}
    st.session_state.q_idx = 0
    st.rerun()

# Active Timed Exam
elif st.session_state.exam_active and not st.session_state.exam_submitted:
    elapsed = time.time() - st.session_state.start_time
    remaining = time_limit - elapsed

    # Auto-submit on expiry
    if remaining <= 0:
        st.session_state.exam_active = False
        st.session_state.exam_submitted = True
        st.warning("🕒 Time has expired. Your test has been submitted automatically.")
        st.rerun()

mins, secs = divmod(int(remaining), 60)

# Timer Header
t_col, p_col = st.columns([1, 2])
with t_col:
    t_color = "red" if remaining < 120 else "green"
    st.markdown(f"<h3 style='color: {t_color}; margin: 0;'>🕒  
{mins:02d}:{secs:02d}</h3>", unsafe_allow_html=True)
with p_col:
    st.caption(f"Question {st.session_state.q_idx + 1} of {len(questions)}")
    st.progress((st.session_state.q_idx + 1) / len(questions))

st.markdown("---")

curr = questions[st.session_state.q_idx]
st.markdown(f"#### {curr['question']}")

chosen = st.session_state.user_choices.get(st.session_state.q_idx)

# Interactive Tap Options
for o_idx, opt_text in enumerate(curr["options"]):
    lbl = f"{chr(65 + o_idx)}. {opt_text}"
    if chosen == o_idx:
        lbl = f"👉 [Selected] {lbl}"

    if st.button(lbl, key=f"q_{st.session_state.q_idx}_opt_{o_idx}",
use_container_width=True):
        st.session_state.user_choices[st.session_state.q_idx] = o_idx
        st.rerun()

```

```

# Navigation Controls
st.write("")
b1, b2, b3 = st.columns([1, 1, 1])
with b1:
    if st.session_state.q_idx > 0 and st.button("⬅ Previous"):
        st.session_state.q_idx -= 1
        st.rerun()
with b2:
    if st.session_state.q_idx < len(questions) - 1 and st.button("Next ➡"):
        st.session_state.q_idx += 1
        st.rerun()
with b3:
    if st.button("Submit Paper 🚩", type="primary"):
        st.session_state.exam_active = False
        st.session_state.exam_submitted = True
        st.rerun()

# Scorecard & Analysis Screen
elif st.session_state.exam_submitted:
    st.header("📊 Performance Scorecard")

    total_q = len(questions)
    correct_count = 0
    wrong_count = 0

    for i, q in enumerate(questions):
        user_ans = st.session_state.user_choices.get(i)
        if user_ans is not None:
            if user_ans == q["correct_index"]:
                correct_count += 1
            else:
                wrong_count += 1

    unattempted_count = total_q - (correct_count + wrong_count)
    marks_per_correct = 2.0
    penalty_per_wrong = 0.666
    net_score = (correct_count * marks_per_correct) - (wrong_count *
penalty_per_wrong)
    max_marks = total_q * marks_per_correct

# Scorecard Metrics
m1, m2, m3, m4 = st.columns(4)
m1.metric("Net Score", f"{net_score:.2f} / {max_marks:.0f}")
m2.metric("Correct (+2)", correct_count)
m3.metric("Wrong (-0.66)", wrong_count)
m4.metric("Unattempted", unattempted_count)

```

```

        accuracy = (correct_count / (correct_count + wrong_count) * 100) if (correct_count +
wrong_count) > 0 else 0
        st.caption(f"***Attempt Accuracy:** {accuracy:.1f}%")

# PDF Download Button
pdf_bytes = generate_pdf_report(
    exam_title, total_q, correct_count, wrong_count, unattempted_count, net_score,
max_marks, questions, st.session_state.user_choices
)
st.download_button(
    label="📄 Download Detailed Scorecard (PDF)",
    data=pdf_bytes,
    file_name=f"Scorecard_{exam_title.replace(' ', '_')}.pdf",
    mime="application/pdf",
    type="primary"
)

st.markdown("---")
st.subheader("Question Solutions & Explanations")

for i, q in enumerate(questions):
    user_ans = st.session_state.user_choices.get(i)
    is_correct = user_ans == q["correct_index"]

    if user_ans is None:
        icon, badge = "🕒", "Unattempted (0.0)"
    elif is_correct:
        icon, badge = "✅", "Correct (+2.0)"
    else:
        icon, badge = "❌", "Incorrect (-0.66)"

    with st.expander(f"{icon} Q{i + 1}: {q['question'][:75]}... [{badge}]"):
        st.write(f"***Question:** {q['question']}")
        for o_idx, opt in enumerate(q["options"]):
            prefix = f"{chr(65 + o_idx)}. "
            if o_idx == q["correct_index"]:
                st.markdown(f"green[**{prefix}{opt} (Correct Answer)**]")
            elif o_idx == user_ans:
                st.markdown(f"red[**{prefix}{opt} (Your Answer)**]")
            else:
                st.write(f"{prefix}{opt}")
        st.info(f"***Explanation:** {q['explanation']}")

if st.button("Take Exam Again"):
    st.session_state.exam_submitted = False
    st.session_state.exam_active = False
    st.session_state.user_choices = {}
    st.session_state.q_idx = 0

```



```
st.rerun()
```